Linux网络--IO多路转接与Reactor系列总览

- Linux 网络编程：从 IO 多路转接到 Reactor
  - 阅读顺序
  - 全系列主线
  - 术语速查

Linux 网络编程：从 IO 多路转接到 Reactor

这组文章沿着一条连续的问题链展开：一个线程如何等待多个连接，如何减少无意义的等待和扫描，如何正确处理可读可写事件，最后如何把底层通知组织成可维护的 Reactor 服务端。

阅读顺序

顺序	文章	回答的问题
01	Linux网络--IO多路转接层01 从阻塞IO到select事件循环	为什么需要多路转接，select 如何完成一次等待
02	Linux网络--IO多路转接层02 poll原理与select服务器改写	poll 如何分离输入输出事件，为什么仍需全量扫描
03	Linux网络--IO多路转接层03 epoll事件模型与内核原理	epoll 如何保存关注集合并只返回就绪事件
04	Linux网络--IO多路转接层04 LT与ET及epoll服务器实现	LT 和 ET 如何通知，epoll 服务器如何正确读写
05	Linux网络--Reactor模式层05 Connection回调与事件分发	如何把 fd、缓冲区、协议和业务组织成 Reactor
06	Linux网络--Reactor模式层06 OneThreadOneLoop与eventfd	如何扩展到多进程、多线程和多个事件循环

全系列主线

```
阻塞 I/O 的等待成本
        |
        v
select：位图表达关注集合，每轮复制并扫描
        |
        v
poll：events/revents 分离，仍复制并扫描数组
        |
        v
```

```
epoll：持久注册 + 回调 + 就绪队列
        |
        v
LT/ET：决定同一个就绪条件要通知几次
        |
        v
Reactor：EventLoop + Connection + Callback
        |
        v
One Thread One Loop：每个线程独立管理自己的连接
```

术语速查

- **fd**：文件描述符。Linux 中 socket、pipe、eventfd 和 epoll 实例都由 fd 标识。
- **就绪**：现在执行读或写有机会立即推进；不等于请求已经完整。
- **LT**：只要条件持续成立，就持续通知。
- **ET**：在就绪状态发生变化时通知。
- **Reactor**：事件循环等待就绪事件，再调用对应对象的回调。
- **OTOL**：One Thread One Loop，每个线程运行一个独立事件循环。。